

Образы являются базовыми строительными блоками для контейнеров и других образов. Когда вы "контейнеризуете" приложение, вы работаете над созданием образа.

Изучив, что такое образы и как их создавать, вы готовы начать использовать контейнеры в своих собственных проектах.

Откуда берутся образы?

При выполнении команды, такой как `docker run hello-world`, Docker автоматически ищет [Docker Hub](#) для образа, если он не найден локально.

Это означает, что мы можем загружать и запускать любой публичный образ с серверов Docker. Например, если мы хотим запустить экземпляр базы данных PostgreSQL, мы можем просто запустить `docker run postgres`, который загрузит и запустит https://hub.docker.com/_/postgres/.

Мы можем искать образы в Docker Hub с помощью `docker search`. Попробуйте запустить `docker search hello-world`.

Поиск находит множество результатов и выводит имя каждого образа, краткое описание, количество звезд и статусы "official" и "automated".

```
$ docker search hello-world
```

NAME	DESCRIPTION	STARS	OFFICIAL	AUTOMATED
hello-world	Hello World!...	1988	[OK]	
kitematic/hello-world-nginx	A light-weig...	153		
tutum/hello-world	Image to tes...	90		[OK]
...				

Давайте рассмотрим список.

Первый результат, `hello-world`, является официальным образом. [Официальные образы](#) курируются и проверяются Docker, Inc. и обычно активно поддерживаются авторами. Они собираются из репозитория в [docker-library](#).

При просмотре результатов поиска CLI вы можете распознать официальный образ по "[OK]" в колонке "OFFICIAL" и также по тому факту, что имя образа не имеет префикса (то есть организации/пользователя). При просмотре Docker Hub страница покажет "Docker Official Images" в качестве репозитория вместо пользователя или организации. Например, см. [страницу Docker Hub](#) образа `hello-world`.

Третий результат, `tutum/hello-world`, отмечен как "automated". Это означает, что образ [автоматически собирается](#) из исходного репозитория. Его [страница Docker Hub](#) показывает предыдущие "Сборки" и ссылку на "Исходный репозиторий" образа (в данном случае, на GitHub), из которого Docker Hub собирает образ.

Второй результат, `kitematic/hello-world-nginx`, не является ни официальным, ни автоматизированным образом. Мы не можем знать, из чего собран образ, так как его [страница Docker Hub](#) не имеет ссылок на какие-либо репозитории. Единственное, что его страница Docker Hub показывает, — это то, что образу 9 лет. Даже если бы в разделе "Overview" образа были ссылки на репозиторий, у нас не было бы гарантий, что опубликованный образ был собран из этого источника.

Также есть другие Docker registry, конкурирующие с Docker Hub, такие как [Quay](#). По умолчанию `docker search` будет искать только в Docker Hub, но для поиска в другом registry вы можете добавить адрес registry перед поисковым запросом, например, `docker search quay.io/hello`. В качестве альтернативы вы можете использовать веб-страницы registry для поиска образов. Взгляните на страницу [образа nordstrom/hello-world на Quay](#). Страница показывает команду для использования, чтобы загрузить образ, что показывает, что мы также можем загружать образы с хостов, отличных от Docker Hub:

```
docker pull quay.io/nordstrom/hello-world
```

Итак, если имя хоста (здесь: `quay.io`) опущено, по умолчанию будет загрузка из Docker Hub.

ПРИМЕЧАНИЕ: Попытка выполнить вышеуказанную команду может завершиться ошибкой с ошибками манифеста, так как тег по умолчанию `latest` отсутствует в образе `quay.io/nordstrom/hello-world`. Указание правильного тега для образа позволит загрузить образ без ошибок, например: `docker pull quay.io/nordstrom/hello-world:2.0`

Детальный взгляд на образ

Давайте вернемся к более актуальному образу, чем 'hello-world', образу Ubuntu, одному из наиболее распространенных Docker образов для использования в качестве базы для вашего собственного образа.

Давайте загрузим Ubuntu и посмотрим на первые строки:

```
$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
```

Поскольку мы не указали тег, Docker по умолчанию использовал `latest`, который обычно является последним собранным и загруженным в registry образом. **Однако** в данном случае [README](#) репозитория говорит, что тег `ubuntu:latest` указывает на "последний LTS" вместо этого, так как это версия, рекомендуемая для общего использования.

Образы могут быть помечены тегами для сохранения разных версий одного и того же образа. Вы определяете тег образа, добавляя `:<tag>` после имени образа.

[Страница Docker Hub](#) Ubuntu показывает, что есть тег с именем 22.04, который гарантирует, что образ основан на Ubuntu 22.04. Давайте загрузим и его:

```
$ docker pull ubuntu:22.04

22.04: Pulling from library/ubuntu
c2ca09a1934b: Downloading [=====>]
34.25MB/38.64MB
d6c3619d2153: Download complete
0efe07335a04: Download complete
6b1bb01b3a3b: Download complete
43a98c187399: Download complete
```

Образы состоят из разных слоев, которые загружаются параллельно для ускорения загрузки. Тот факт, что образы состоят из слоев, также имеет другие аспекты, и мы поговорим о них в части 3.

Мы также можем локально помечать образы для удобства, например, `docker tag ubuntu:22.04 ubuntu:jammy_jellyfish` создает тег `ubuntu:jammy_jellyfish`, который ссылается на `ubuntu:22.04`.

Тегирование также является способом "переименовать" образы. Запустите `docker tag ubuntu:22.04 fav_distro:jammy_jellyfish` и проверьте `docker image ls`, чтобы увидеть, какие эффекты имела команда.

Подводя итог, имя образа может состоять из 3 частей плюс тег. Обычно следующим образом: `registry/organisation/image:tag`. Но может быть таким коротким, как `ubuntu`, тогда registry по умолчанию будет Docker hub, organisation — *library*, а тег — *latest*. Организация также может быть пользователем, но называть ее организацией может быть понятнее.

Упражнения 1.5 - 1.6

:::info Упражнение 1.5: Размеры образов

В [Упражнении 1.3](#) мы использовали `devopsdockeruh/simple-web-service:ubuntu`.

Вот то же приложение, но вместо Ubuntu использует [Alpine Linux](#): `devopsdockeruh/simple-web-service:alpine`.

Загрузите оба образа и сравните их размеры. Зайдите внутрь контейнера Alpine и убедитесь, что функциональность секретного сообщения такая же. Версия Alpine не имеет `bash`, но имеет `sh`, более простую оболочку.

:::

:::info Упражнение 1.6: Привет Docker Hub

Запустите `docker run -it devopsdockeruh/pull_exercise`.

Команда будет ждать вашего ввода.

Перейдите через [Docker hub](#), чтобы найти документацию и Dockerfile, который использовался для создания образа.

Прочитайте Dockerfile и/или документацию, чтобы узнать, какой ввод заставит приложение ответить "секретным сообщением".

Отправьте секретное сообщение и команду(ы), использованные для его получения, в качестве ответа.

:::

Сборка образов

Наконец, мы переходим к созданию собственных образов и поговорим о [Dockerfile](#) и почему это так здорово.

Dockerfile — это просто файл, который содержит инструкции для сборки образа. Вы определяете, что должно быть включено в образ, с помощью различных инструкций. Мы узнаем о лучших практиках здесь, создав один.

Давайте возьмем самое простое приложение и контейнеризуем его. Вот скрипт с названием "hello.sh"

hello.sh

```
#!/bin/sh

echo "Hello, docker!"
```

Сначала мы проверим, что он вообще работает. Создайте файл, добавьте права на выполнение и запустите его:

```
$ chmod +x hello.sh

$ ./hello.sh
Hello, docker!
```

- Если вы используете Windows, вы можете пропустить эти два шага и добавить `chmod +x hello.sh` в Dockerfile.

А теперь создадим из него образ. Нам нужно создать `Dockerfile`, который объявляет все необходимые зависимости. По крайней мере, он зависит от чего-то, что может запускать shell скрипты. Мы выберем [Alpine](#), небольшой дистрибутив Linux, который часто используется для создания небольших образов.

Хотя здесь мы используем Alpine, вы можете использовать Ubuntu во время упражнений. Образы Ubuntu по умолчанию содержат больше инструментов для отладки того, что не работает, когда что-то идет не так. В части 3 мы поговорим больше о том, почему небольшие образы важны.

Мы выберем конкретную версию данного образа, которую хотим использовать. Это гарантирует, что мы не случайно обновимся через критическое изменение, и мы знаем, какие образы нужно обновлять, когда в старых образах есть известные уязвимости безопасности.

Теперь создайте файл и назовите его "Dockerfile" и поместите в него следующие инструкции:

Dockerfile

```
# Start from the alpine image that is smaller but no fancy tools
FROM alpine:3.19

# Use /usr/src/app as our workdir. The following instructions will be executed in
this location.
WORKDIR /usr/src/app

# Copy the hello.sh file from this directory to /usr/src/app/ creating
/usr/src/app/hello.sh
COPY hello.sh .

# Alternatively, if we skipped chmod earlier, we can add execution permissions
during the build.
# RUN chmod +x hello.sh

# When running docker run the command will be ./hello.sh
CMD ./hello.sh
```

Отлично! Мы можем использовать команду [docker build](#), чтобы превратить Dockerfile в образ.

По умолчанию `docker build` будет искать файл с именем Dockerfile. Теперь мы можем запустить `docker build` с указанием, где собирать (`.`) и дать ему имя (`-t <name>`):

```
$ docker build . -t hello-docker
=> [internal] load build definition from Dockerfile
0.0s
=> => transferring dockerfile: 478B
0.0s
=> [internal] load metadata for docker.io/library/alpine:3.19
2.1s
=> [auth] library/alpine:pull token for registry-1.docker.io
0.0s
=> [internal] load .dockerignore
0.0s
=> => transferring context: 2B
0.0s
=> [1/3] FROM
docker.io/library/alpine:3.19@sha256:c5b1261d6d3e43071626931fc004f70149baeba2c8ec672bd4
0.0s
=> [internal] load build context
0.0s
=> => transferring context: 68B
0.0s
=> [2/3] WORKDIR /usr/src/app
0.0s
=> [3/3] COPY hello.sh .
0.0s
=> exporting to image
0.0s
=> => exporting layers
0.0s
=> => writing image
sha256:5f8f5d7445f34b0bcfaaa4d685a068cdccc1ed79e65068337a3a228c79ea69c8
0.0s
=> => naming to docker.io/library/hello-docker
```

Давайте убедимся, что образ существует:

```
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-docker	latest	5f8f5d7445f3	4 minutes ago	7.73MB

:::tip Отказ в разрешении

Если вы получаете `"/bin/sh: ./hello.sh: Permission denied"`, это потому, что `chmod +x hello.sh` был пропущен ранее. Вы можете просто раскомментировать инструкцию RUN между инструкциями COPY и CMD

:::

:::tip not found

Если вы получаете `"/bin/sh: ./hello.sh: not found"` и вы используете Windows, это может быть потому, что по умолчанию Windows использует [CRLF](#) как окончание строки. Unix, в нашем случае Alpine, использует только LF, что делает копирование нашего `hello.sh` невалидным bash скриптом на этапе сборки. Чтобы преодолеть эту ошибку, измените окончания строк на LF перед запуском `docker build`

:::

Теперь выполнение приложения так же просто, как запуск `docker run hello-docker`. Попробуйте!

Во время сборки мы видим из вывода, что есть три шага: [1/3], [2/3] и [3/3]. Шаги здесь представляют [слои](#) образа, так что каждый шаг — это новый слой поверх базового образа (alpine:3.19 в нашем случае).

Слои имеют множество функций. Мы часто пытаемся ограничить количество слоев, чтобы экономить место для хранения, но слои могут работать как кэш во время сборки. Если мы просто редактируем последние строки Dockerfile, команда build может начать с предыдущего слоя и пропустить прямо к разделу, который изменился. COPY автоматически обнаруживает изменения в файлах, так что если мы изменим hello.sh, он запустится с шага 3/3, пропуская 1 и 2. Это можно использовать для создания более быстрых конвейеров сборки. Мы поговорим больше об оптимизации в части 3.

Также возможно вручную создавать новые слои поверх образа. Давайте теперь создадим новый файл с именем `additional.txt` и скопируем его внутрь контейнера.

Нам понадобятся два терминала, которые будут называться 1 и 2 в следующих списках. Давайте начнем с запуска образа:

```
# do this in terminal 1
$ docker run -it hello-docker sh
/usr/src/app #
```

Теперь мы внутри контейнера. Мы заменили CMD, который мы определили ранее, на `sh` и использовали `-i` и `-t`, чтобы запустить контейнер, чтобы мы могли взаимодействовать с ним.

Во втором терминале мы скопируем файл внутрь контейнера:

```
# do this in terminal 2
$ docker ps
CONTAINER ID   IMAGE          COMMAND        CREATED        STATUS        PORTS
NAMES
9c06b95e3e85   hello-docker   "sh"          4 minutes ago  Up 4 minutes
zen_rosalind

$ touch additional.txt
$ docker cp ./additional.txt zen_rosalind:/usr/src/app/
```

Файл создается с помощью команды `touch` прямо перед копированием.

Давайте убедимся, что файл скопирован внутрь контейнера:

```
# do this in terminal 1
/usr/src/app # ls
additional.txt  hello.sh
```

Отлично! Теперь мы внесли изменение в контейнер. Мы можем использовать команду [docker diff](#), чтобы проверить, что изменилось

```
# do this in terminal 2
$ docker diff zen_rosalind
C /usr
C /usr/src
C /usr/src/app
A /usr/src/app/additional.txt
C /root
A /root/.ash_history
```

Символ перед именем файла указывает тип изменения в файловой системе контейнера: A = добавлено, D = удалено, C = изменено. additional.txt был создан, а наш `ls` создал `.ash_history`.

Далее мы сохраним изменения как *новый образ* с помощью команды [docker commit](#):

```
# do this in terminal 2
$ docker commit zen_rosalind hello-docker-additional
sha256:2f63baa355ce5976bf89fe6000b92717f25dd91172aed716208e784315bfc4fd
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-docker-additional	latest	2f63baa355ce	3 seconds ago	7.73MB
hello-docker	latest	444f21cf7bd5	31 minutes ago	7.73MB

Технически команда `docker commit` добавила новый слой поверх образа `hello-docker`, и результирующий образ получил имя `hello-docker-additional`.

Мы фактически не будем использовать команду `docker commit` снова в течение этого курса. Это потому, что определение изменений в Dockerfile является гораздо более устойчивым методом управления изменениями. Никаких магических действий или скриптов, только Dockerfile, который можно контролировать версиями.

Давайте сделаем именно это и создадим `hello-docker` с тегом `v2`, который включает файл `additional.txt`. Новый файл можно добавить с помощью инструкции [RUN](#):

Dockerfile

```
# Start from the alpine image
FROM alpine:3.19

# Use /usr/src/app as our workdir. The following instructions will be executed in
this location.
WORKDIR /usr/src/app

# Copy the hello.sh file from this location to /usr/src/app/ creating
```

```
/usr/src/app/hello.sh.  
COPY hello.sh .  
  
# Execute a command with `/bin/sh -c` prefix.  
RUN touch additional.txt  
  
# When running Docker run the command will be ./hello.sh  
CMD ./hello.sh
```

Теперь мы использовали инструкцию RUN для выполнения команды `touch additional.txt`, которая создает файл внутри результирующего образа. Практически все, что может быть выполнено в контейнере на основе созданного образа, может быть указано для выполнения с помощью инструкции RUN во время сборки Dockerfile.

Соберите Dockerfile с помощью `docker build . -t hello-docker:v2`, и мы закончили! Давайте сравним вывод ls:

```
$ docker run hello-docker-additional ls  
additional.txt  
hello.sh  
  
$ docker run hello-docker:v2 ls  
additional.txt  
hello.sh
```

Теперь мы знаем, что все инструкции в Dockerfile **кроме** CMD (и еще одной, о которой мы скоро узнаем) выполняются во время сборки. **CMD** выполняется, когда мы вызываем `docker run`, если только мы не перезаписываем его.

Упражнения 1.7 - 1.8

:::info Упражнение 1.7: Образ для скрипта

Мы можем улучшить наши предыдущие решения теперь, когда мы знаем, как создавать и собирать Dockerfile.

Давайте теперь вернемся к [Упражнению 1.4](#).

Создайте новый файл `script.sh` на вашей локальной машине со следующим содержимым:

```
while true  
do  
    echo "Input website:"  
    read website; echo "Searching.."  
    sleep 1; curl http://$website  
done
```

Создайте Dockerfile для нового образа, который начинается с `ubuntu:22.04` и добавьте инструкции для установки `curl` в этот образ. Затем добавьте инструкции для копирования файла скрипта в этот образ и, наконец, установите его для запуска при старте контейнера с помощью CMD.

После того, как вы заполнили Dockerfile, соберите образ с именем "curler".

- Если вы получаете отказ в разрешении, используйте `chmod`, чтобы дать разрешение на запуск скрипта.

Следующее теперь должно работать:

```
$ docker run -it curler

Input website:
helsinki.fi
Searching..
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="https://www.helsinki.fi/">here</a>.</p>
</body></html>
```

Помните, что [RUN](#) можно использовать для выполнения команд во время сборки образа!

Отправьте Dockerfile.

...

...info Упражнение 1.8: Двухстрочный Dockerfile

По умолчанию наш `devopsdockeruh/simple-web-service:alpine` не имеет CMD. Вместо этого он использует *ENTRYPOINT* для объявления, какое приложение запускается.

Мы поговорим больше о *ENTRYPOINT* в следующем разделе, но вы уже знаете, что последний аргумент в `docker run` можно использовать для передачи команды или аргумента.

Как вы, возможно, заметили, он не запускает веб-сервис, несмотря на название "simple-web-service". Нужен подходящий аргумент, чтобы запустить сервер!

Попробуйте `docker run devopsdockeruh/simple-web-service:alpine hello`. Приложение читает аргумент "hello", но сообщит, что hello не принимается.

В этом упражнении создайте Dockerfile и используйте FROM и CMD, чтобы создать совершенно новый образ, который автоматически запускает `server`.

Документация Docker [CMD](#) косвенно говорит, что если образ имеет определенный ENTRYPOINT, CMD используется для определения аргументов по умолчанию.

Пометьте новый образ как "web-server"

Верните Dockerfile и команду, которую вы использовали для запуска контейнера.

Запуск собранного образа "web-server" должен выглядеть так:

```
$ docker run web-server
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery
middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in
```

```
production.  
- using env:  export GIN_MODE=release  
- using code: gin.SetMode(gin.ReleaseMode)  
  
[GIN-debug] GET    /*path          --> server.Start.func1 (3 handlers)  
[GIN-debug] Listening and serving HTTP on :8080
```

- У нас пока нет способа доступа к веб-сервису. Поэтому подтверждение того, что вывод консоли такой же, будет достаточным.
- Название упражнения может быть полезной подсказкой здесь.

...